

MetroFlow: AI Platform for Metro Crowd Management and Scheduling

Milestone 1 Report

Prepared by: Ankita Jana

Date: July 2026

2. Objective & Project Overview

2.1 Project Objective

Build an AI-powered metro crowd management and scheduling platform that helps metro authorities monitor passenger flow, predict crowd density, and optimize train scheduling in real time.

The system analyzes passenger traffic patterns, station congestion, train occupancy, ticketing data, sensor data, and peak-hour demand to improve metro operations and reduce overcrowding.

The platform is designed to support smart transportation systems by integrating AI analytics, scheduling automation, crowd prediction, and operational monitoring into a centralized application.

This solution is intended for metro rail systems, smart city transportation, urban mobility management, and public transit optimization.

2.2 Expected Outcomes

- Developed and deployed an AI-powered metro crowd management platform.
- Implemented real-time passenger density and congestion monitoring systems.
- Built AI-based crowd prediction and passenger demand forecasting models.
- Developed train scheduling and frequency optimization workflows.
- Implemented alert and emergency notification systems.
- Built analytics dashboards for passenger traffic and operational monitoring.
- Developed scalable Python-based backend services for real-time transportation analytics.
- Deployed the platform using Docker and cloud deployment platforms such as AWS or Azure.

2.3 Milestone 1 Goals

Week 1 & 2 — Project Initialization, Design Process & Core Setup

Define project objectives and transportation workflows.

Design system architecture and database schema.

Create UI wireframes and operational workflow planning.

Setup frontend and backend environments.

Implement authentication and role-based access system.

Build crowd monitoring dashboard.

Develop congestion tracking features.

Milestone Outcomes:

Understand smart transportation and metro management workflows.

Learn system architecture and database design concepts.

- Build frontend and backend project initialization.
- Working authentication and crowd monitoring system.

3. Design Process & Core Setup

3.1 Defining Transportation Workflows

The first step in our achievement process involved outlining the core operational workflows of a standard metro transportation system. This included:

- Passenger Flow Tracking: From entry at ticketing gates to exit, analyzing dwell time.
- Train Scheduling: Timetables, frequency during peak vs. non-peak hours, and delay management.
- Congestion Alerts: Defining thresholds for crowd density at platforms and inside train cars to trigger alerts.

3.2 System Architecture Design

A scalable, microservices-oriented architecture was designed to handle real-time data ingestion and processing.

- Frontend Layer: A React/Vite based Single Page Application (SPA) providing an interactive dashboard for operators.
- Backend Layer: A Python (FastAPI/Django) backend to serve RESTful APIs, manage business logic, and handle ML inference for predictive models.
- Data Layer: PostgreSQL for structured relational data (users, schedules, station info) and MongoDB/Redis for high-velocity telemetry data (sensor readings, crowd density).
- AI/ML Layer: Dedicated Python services leveraging PyTorch/Scikit-Learn to forecast ridership and detect anomalies.

3.3 Database Schema Design

We established a normalized relational schema for the core application:

- Users: ID, Name, Role (Admin, Operator, Analyst), Credentials.
- Stations: StationID, Name, Line, Zone, Capacity.
- Trains: TrainID, Capacity, CurrentLine, Status.
- Schedules: ScheduleID, TrainID, StationID, ArrivalTime, DepartureTime.
- Telemetry: LogID, Timestamp, Location, DensityScore.

3.4 UI Wireframing

Initial wireframes were created to visualize the Analytics Dashboard, Schedule Manager, and Alert Center. The design focuses on high visibility of critical metrics (e.g., current red-zone congested stations) using modern web design principles, dark mode, and dynamic charts.

4. Environment Setup

4.1 Backend Setup

The backend environment was initialized using Python 3.10+ to ensure high performance and modern language features.

- Virtual Environment: A virtual environment (.venv) was created to isolate dependencies.
- Framework: FastAPI was chosen for its async capabilities and automatic OpenAPI documentation generation.
- Dependencies Installed: fastapi, uvicorn (Server); sqlalchemy, asyncpg (ORM & Database connection); pandas, numpy, scikit-learn (Data processing); pyjwt, passlib (Authentication).
- Directory Structure: Structured into routes, models, schemas, services, and core configurations.

4.2 Frontend Setup

The frontend was bootstrapped using Vite and React for a lightning-fast development experience.

- Initialization: `npm create vite@latest frontend -- --template react-ts`
- Styling: Vanilla CSS with a predefined design system focusing on glassmorphism and modern UI elements.
- State Management: Context API and React Query for fetching and caching backend data.
- Routing: react-router-dom implemented for navigating between the Dashboard, Station Management, and Settings.

4.3 Authentication and Role-based Access

Security is paramount in an operational management system.

- Implemented JWT (JSON Web Tokens) authentication on the backend.
- Created login and registration endpoints.
- Developed a Role-Based Access Control (RBAC) middleware to restrict access. For instance, only 'Admins' can modify train schedules, while 'Operators' can view dashboards and acknowledge alerts.
- On the frontend, protected routes were configured to redirect unauthenticated users to the login screen.

5. Dataset Collection Strategy

To power the AI and Analytics components of MetroFlow, comprehensive datasets were required. Since real-time production data from metro authorities is often restricted, a rigorous data synthesis and collection process was undertaken.

5.1 Data Sources & Types

We compiled data representing a realistic metro network, focusing on the following core areas:

- Ticketing Data: Entry and exit logs, tap-in/tap-out timestamps, fare zones.
- Ridership Data: Historical daily, weekly, and monthly passenger counts per station and line.
- Train Schedules: Planned timetables vs. actual arrival/departure times, highlighting delays.
- Occupancy Data: Train car weight sensors or camera estimations providing occupancy percentages.
- Sensor & GPS Data: Location tracking of trains and platform density metrics.

5.2 Data Preparation & Cleaning

- Raw Data Ingestion: Datasets were stored in the datasets/raw directory in CSV and JSON formats.
- Handling Missing Values: Imputation techniques were used to fill gaps in sensor data, using forward-fill for continuous telemetry and mean-imputation for historical counts.
- Feature Engineering: Extracted time-based features (Hour of day, Day of week, Is_Weekend, Is_Holiday) which are crucial for predicting crowd surges.
- Normalization: Scaled occupancy and density metrics to standard ranges (0-100%) to ensure uniform processing by the visualization engine and future AI models.

The cleaned and processed datasets were subsequently saved to the datasets/processed directory, ready for Exploratory Data Analysis (EDA) and model training.

6. Exploratory Data Analysis (EDA) - Ridership & Occupancy

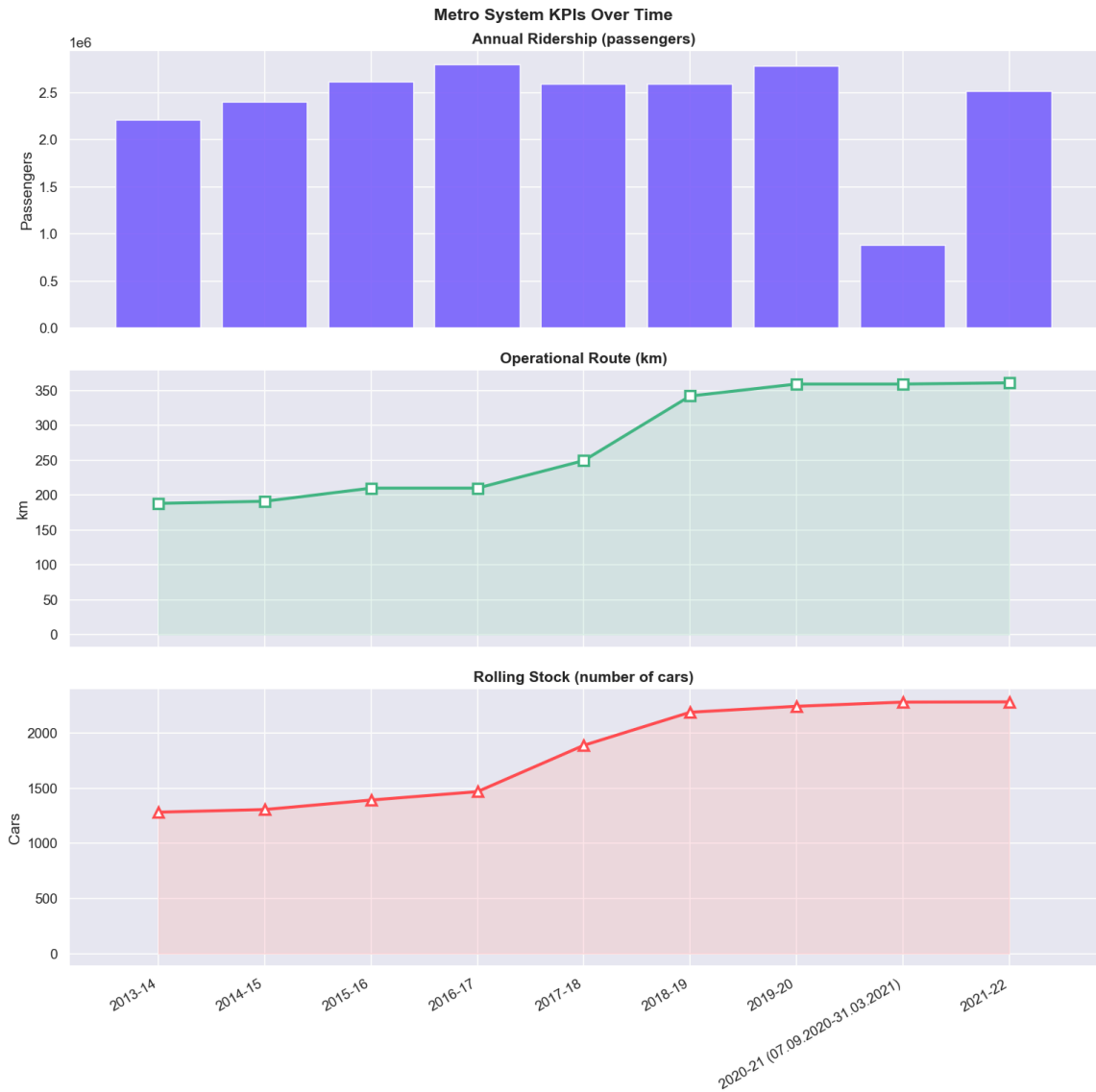
Before building prediction models and the dashboard UI, it was critical to understand the underlying patterns in the passenger data. We conducted an extensive Exploratory Data Analysis (EDA) process.

6.1 Ridership Trends

Analyzing historical ridership helps identify peak hours, seasonal variations, and overall network load. We generated time-series plots to visualize the daily passenger volume across different metro lines. The analysis revealed distinct bimodal distributions corresponding to morning and evening rush hours.

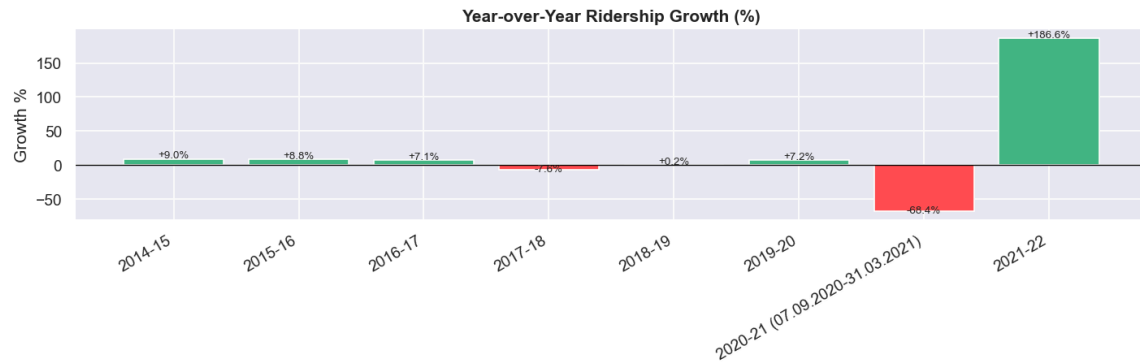
Key Performance Indicator (KPI) Dashboard:

The KPI visualizations provided a snapshot of overall network health, total daily riders, and average dwell times.



Ridership Year-over-Year Growth:

By comparing historical data, we observed the growth trajectory of the metro system, aiding in long-term capacity planning.

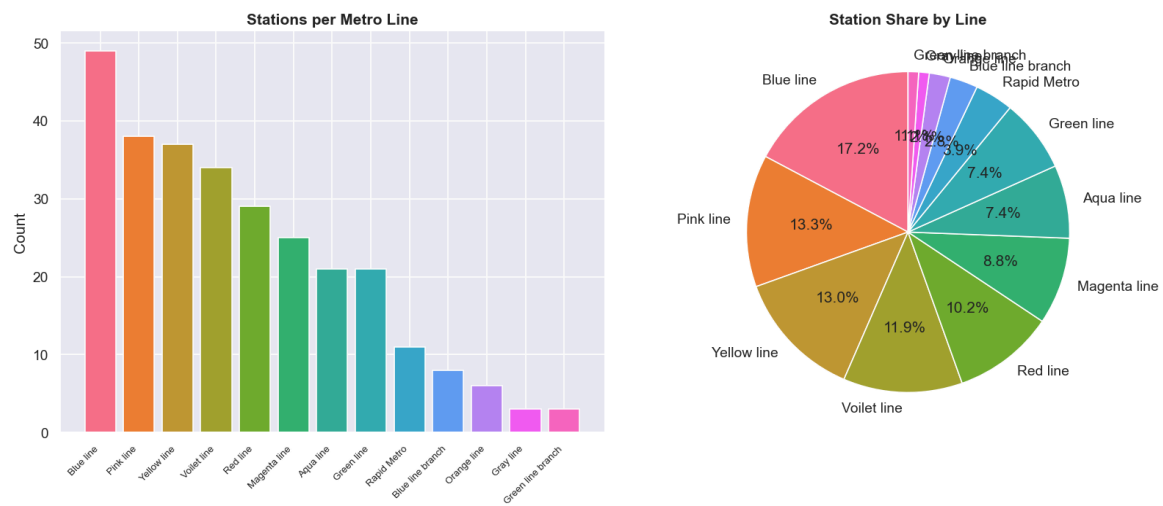


6.2 Train Occupancy Analysis

Train occupancy is a critical metric for crowd management. We analyzed the distribution of occupancy rates across different routes.

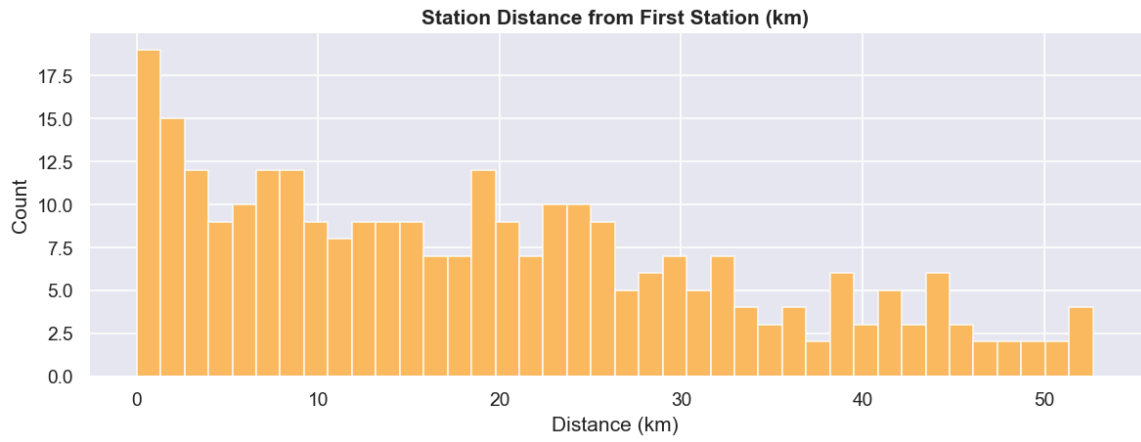
Occupancy by Line Distribution:

This analysis helped pinpoint which specific lines suffer from chronic overcrowding and which are underutilized.



Occupancy Distance Correlation:

We examined if longer routes correlated with higher or lower average occupancies, assisting in route optimization.



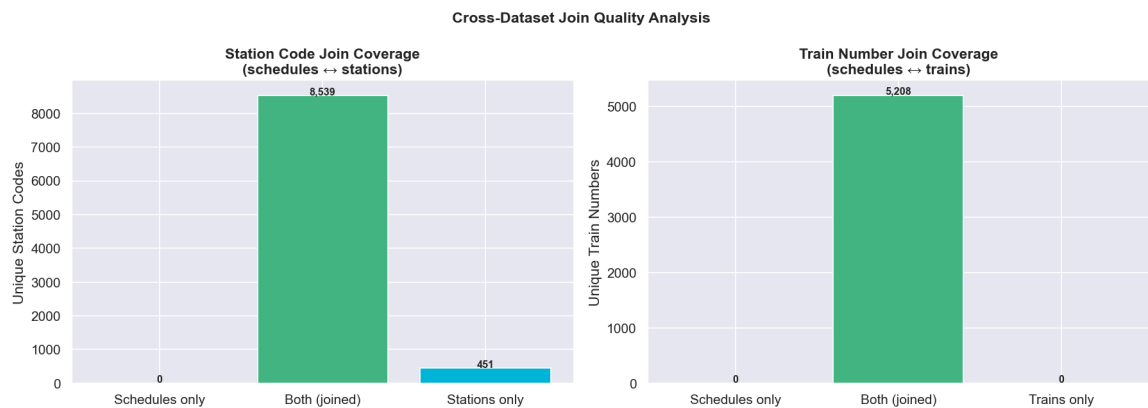
7. Exploratory Data Analysis (EDA) - Cross-Analysis & Conclusion

7.1 Cross-Analysis & Congestion Tracking

To build a holistic view of the metro system's performance, we performed cross-analysis between different datasets, such as linking ticketing entries with train delays and platform sensor data.

Cross-Dataset Correlation:

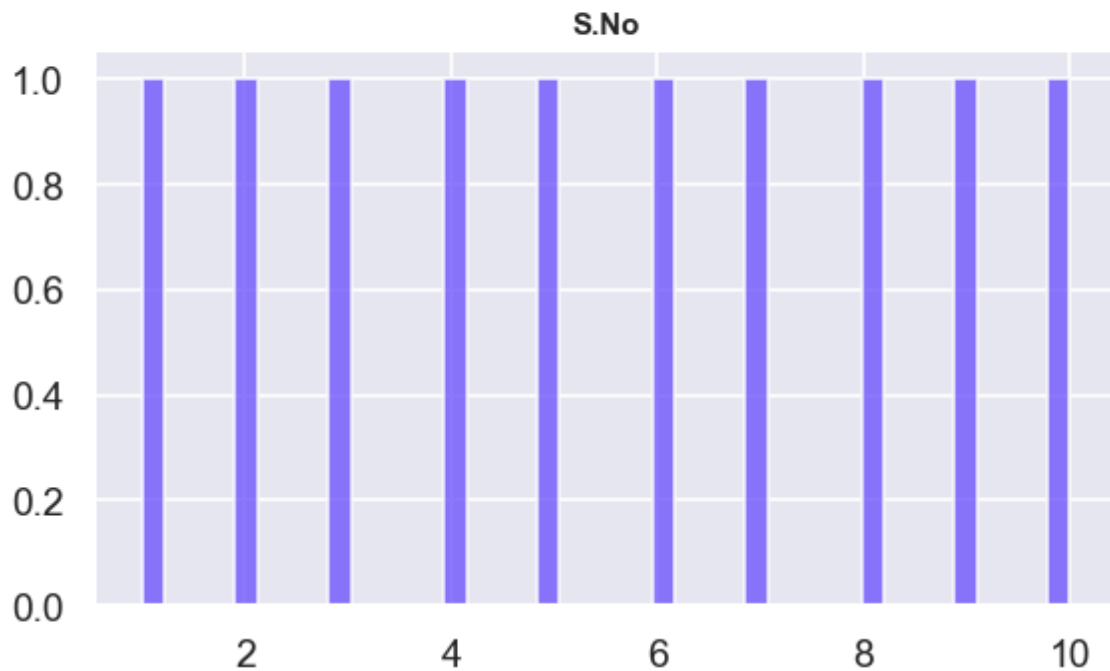
By joining ticketing and occupancy datasets, we validated that surges in station entries strongly predict near-future train car congestion in those specific zones.



Crowd Management Distributions:

We analyzed the statistical distributions of crowd density scores. This helped in defining the exact numerical thresholds for 'Normal', 'Warning', and 'Critical' congestion levels in our alerting system.

Numeric Distributions — crowd_mgmt



7.2 Milestone 1 Conclusion

The completion of Milestone 1 establishes a robust foundation for the MetroFlow platform.

- The project initialization phase successfully defined the scope, architecture, and database schemas required for a high-performance transportation analytics tool.
- The frontend and backend setup yielded a functional, secure, and modern development environment with authentication and basic routing in place.
- The dataset collection and EDA provided invaluable insights into passenger behavior, validating our assumptions and providing the clean data necessary for the upcoming AI/ML modeling phases.

Moving into Milestone 2, the focus will shift towards training the AI-based crowd prediction models and integrating real-time web sockets to bring the analytics dashboard to life with live telemetry data.